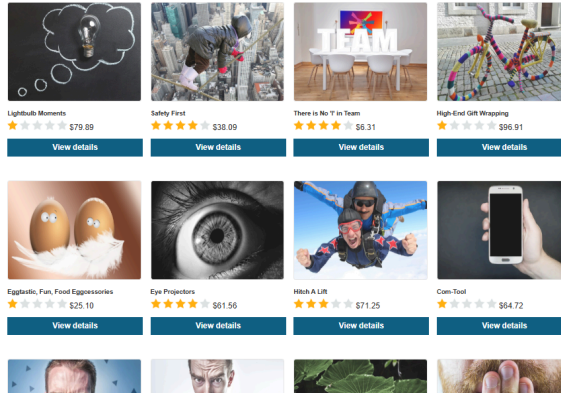


WE LIKE TO SHOP



Actividad 12

File path traversal, validation of file extension with null byte bypass

Guerrero Hurtado Lizbeth

Marco Teórico

¿Qué es File Path Traversal?

Es una vulnerabilidad de control de acceso que ocurre cuando una aplicación web utiliza entradas no sanitizadas del usuario para construir rutas del sistema de archivos. Permite a un atacante salir del directorio web público (webroot) mediante secuencias de escape y acceder a archivos arbitrarios alojados en el servidor subyacente.

Validation of File Extension with Null Byte Bypass:

En este escenario, la aplicación web intenta protegerse comprobando que el parámetro recibido termine obligatoriamente con una extensión de archivo válida (por ejemplo, .jpg o .png). Sin embargo, si la aplicación web está desarrollada en un lenguaje de alto nivel (como PHP o Java en versiones antiguas) o interactúa con APIs en C/C++ a nivel de sistema operativo, es susceptible a la inyección de un byte nulo (%00 en codificación URL o 0x00 en hexadecimal). El byte nulo actúa como el delimitador tradicional de fin de cadena (Null Terminator) en C/C++.

Mecanismo de Explotación:

El fallo conceptual radica en la discrepancia de cómo procesan la cadena la lógica de la aplicación web y la API de lectura de archivos del sistema operativo:

Validación de la Aplicación: La lógica de alto nivel (o expresión regular) analiza la cadena completa ../../etc/passwd%00.jpg, observa que termina en .jpg y aprueba la solicitud.

Lectura en el Sistema Operativo: Al pasar la cadena a la función subyacente del sistema (como fopen() en C), el compilador/OS encuentra el byte nulo (%00), interpreta que la cadena de texto ha finalizado ahí y trunca todo lo que esté a la derecha.

Payload enviado: ../../etc/passwd%00.jpg

Validación de Extensión (Aplicación web): ¿Termina en .jpg? → Sí (Pasa el filtro)

Ejecución en OS (Truncamiento por Byte Nulo): /etc/passwd%00.jpg → Lee: /etc/passwd

Punto Clave de Seguridad:

Validar sufijos o extensiones mediante comprobaciones simples de cadenas es ineficaz si la aplicación no desinfecta caracteres de control binarios como %00 antes de pasar la entrada a las

funciones de I/O de archivos. La disparidad en el manejo de delimitadores de cadena entre capas de software permite la evasión total de los controles de validación.

Impacto en la Tríada CIA:

Confidencialidad (Alto): Compromiso crítico del pilar. Permite exfiltrar archivos sensibles del sistema operativo (/etc/passwd), credenciales de base de datos en archivos .env o configuraciones, claves privadas SSH y código fuente de la aplicación.

Integridad (Bajo): En este laboratorio la vulnerabilidad es de lectura, pero las credenciales obtenidas podrían utilizarse para modificar información en fases posteriores.

Disponibilidad (Medio): La lectura masiva o repetida de dispositivos especiales del sistema (/dev/urandom, /dev/zero) mediante esta técnica puede agotar recursos del servidor y derivar en una Denegación de Servicio (DoS)..

Procedimiento de Explotación (Metodología en Burp Suite)

Reconocimiento y Captura de Petición:

- Abrir el navegador integrado de Burp Suite y acceder a la tienda virtual del laboratorio.
- Seleccionar cualquier producto para forzar a la aplicación a cargar una imagen.
- Ir a la pestaña Proxy > HTTP history y ubicar la solicitud GET de la imagen:

GET /image?filename=72.jpg HTTP/1.1

Envío al Módulo Repeater:

- Hacer clic derecho sobre la petición y seleccionar Send to Repeater (o presionar Ctrl + R).

Prueba de Bloqueo por Extensión (Comprobación del Filtro):

- Intentar enviar un payload estándar como ?filename=../../../../etc/passwd.
- Observar que la aplicación devuelve un mensaje de error o no encuentra el recurso, lo que confirma que el backend valida que la entrada de texto finaliza con una extensión de imagen permitida (ej. .jpg).

Inyección del Payload con Byte Nulo (Bypass de Extensión):

- Construir el payload incluyendo las secuencias de escape ../, el archivo objetivo

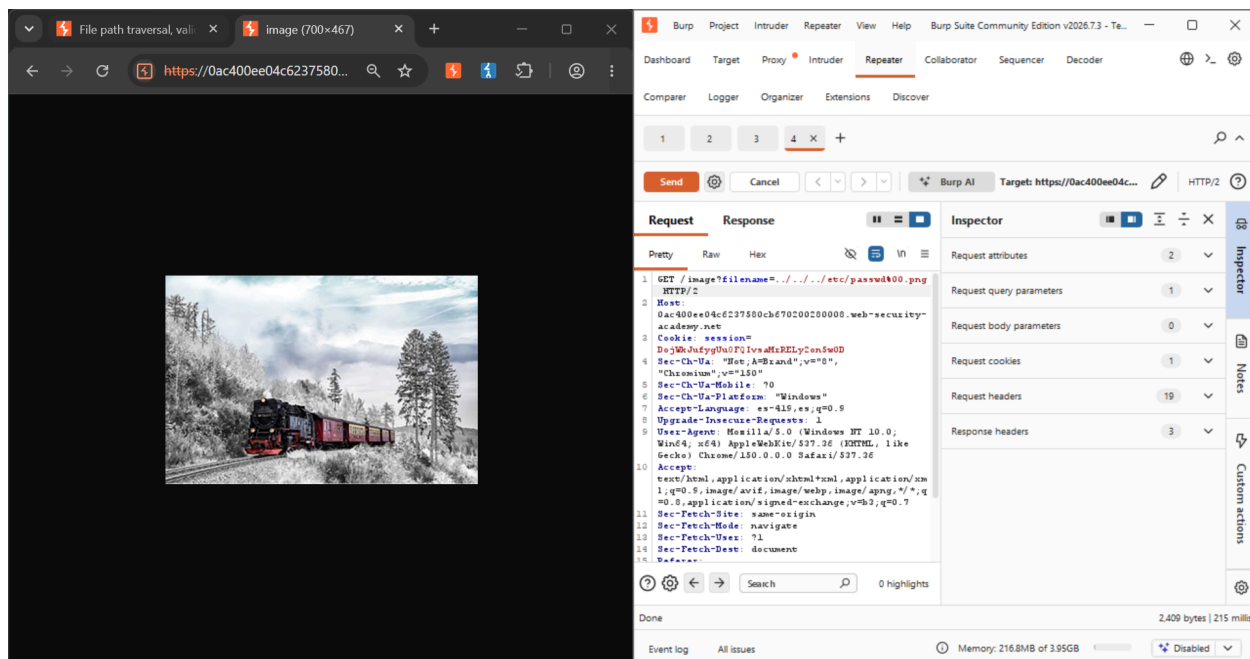
/etc/passwd, el byte nulo codificado %00 y la extensión exigida .jpg.

- Modificar la línea de la solicitud en Repeater para que quede estructurada así:

```
GET /image?filename=../../etc/passwd%00.jpg HTTP/1.1
```

Exfiltración y Confirmación:

- Hacer clic en el botón Send.
- Verificar que la respuesta del servidor devuelva un código de estado HTTP/2 200 OK.
- Confirmar en el cuerpo de la respuesta (Response Body) el contenido del archivo /etc/passwd.
- Refrescar el navegador principal para validar la actualización del banner a "Lab Solved".



Estrategias de Mitigación Recomendadas

Actualización del Entorno de Ejecución (Runtime/Lenguaje):

Asegurar el uso de versiones modernas de lenguajes de programación (por ejemplo, PHP 5.3.4+ o versiones actuales de Java/Node.js) que han sido parcheadas nativamente para neutralizar e ignorar los ataques de truncamiento por byte nulo en funciones del sistema de archivos.

Sanitización Estricta de la Entrada:

Eliminar o codificar cualquier carácter nulo (%00, \0, 0x00) o de control de la cadena enviada

por el usuario antes de procesarla en la lógica de la aplicación.

Verificación de Ruta Canónica (Canonicalization Check):

Resolver la ruta absoluta canónica en el servidor y validar de forma programática que empiece estrictamente dentro del directorio base destinado para las imágenes.

Ejemplo en Java:

```
// Sanitizar entrada y resolver ruta

String sanitizedInput = userInput.replace("\0", "");

File file = new File(BASE_DIRECTORY, sanitizedInput);

String canonicalPath = file.getCanonicalPath();

if (!canonicalPath.startsWith(BASE_DIRECTORY)) {

    throw new SecurityException("Intento de Path Traversal detectado.");

}
```

Uso de Claves Indirectas o Mapeo en BD:

No aceptar nombres ni extensiones de archivos a través de parámetros URL. Se deben utilizar identificadores numéricos mapeados en base de datos (ej. GET /image?id=72).

Principio de Menor Privilegio:

Configurar el usuario del servicio web (www-data) con permisos de lectura sumamente restringidos en el sistema operativo para evitar el acceso a /etc/passwd u otros directorios del sistema.

Conclusiones y Aprendizaje

- **Incompatibilidad entre capas de Software:** Este laboratorio demuestra cómo las discrepancias en el manejo de datos entre lenguajes de alto nivel y funciones de bajo nivel en C/C++ generan vulnerabilidades críticas de elusión de seguridad.
- **Inseguridad de las Validaciones de Sufijo:** Validar simplemente que una cadena termine en .jpg no es suficiente si la cadena contiene caracteres de control que alteran su procesamiento posterior a nivel de sistema operativo.

-
- **Necesidad de Controles Canónicos:** La prevención de Path Traversal no debe basarse en verificar extensiones de archivo, sino en desinfectar caracteres binarios, validar rutas canónicas y limitar el acceso mediante el principio de menor privilegio.